

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

УДК 004.42

УТВЕРЖДАЮ
Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель департамента
программной инженерии
Н.А. Павлочев
«_____» _____ 2026 г.

Выпускная квалификационная работа

на тему «Андроид-приложение для совместного планирования поездок»

по направлению подготовки 09.03.04 «Программная инженерия»

Научный руководитель
Доцент департамента программной
инженерии факультета компьютерных наук
Брейман Александр Давидович

Подпись, Дата

Выполнил
студент группы БПИ223
образовательной программы
«Программная инженерия»
Новик Илья Иванович

Подпись, Дата

Москва 2026

Содержание

Реферат	3
Abstract	4
Основные определения, термины и сокращения	5
1 Введение	6
2 Предметная область и существующие решения	8
2.1 Предметная область и актуальность	8
2.2 Описание существующих решений	8
2.2.1 Splitwise	8
2.2.2 Wanderlog	8
2.2.3 TravelSpend	9
2.2.4 TripIt	9
2.2.5 Яндекс Путешествия	9
2.2.6 RUSSPASS	9
2.3 Описание разрабатываемого решения	9
2.4 Сравнительный анализ существующих решений	10
2.5 Функциональный разрыв и обоснование ниши разрабатываемого приложения	11
2.6 Выводы по главе	11
3 Проектирование системы	12
3.1 Пользовательские сценарии	12
3.2 Архитектура системы	13
3.2.1 Архитектура клиентской части	13
3.2.2 Архитектура серверной части	14
3.2.3 Взаимодействие клиента и сервера	14
3.3 Модель данных и связи сущностей	15
3.4 Подсистемы и функциональное назначение	15
3.5 Выбор методов и средств реализации	16
3.5.1 Серверная часть	17
3.5.2 Хранилище данных	17
3.5.3 Клиентская часть	17
3.5.4 Интеграционные компоненты	18
3.5.5 Обоснование выбранных паттернов	18
3.6 Нефункциональные решения	19
3.7 Выводы по главе	20

4	Программная реализация системы	21
4.1	Реализация подсистемы авторизации и профиля	22
4.1.1	Реализация на клиенте	22
4.1.2	Реализация на сервере	22
4.1.3	Пользовательский поток (владелец/участник)	22
4.1.4	Результат для пользователя	23
4.2	Реализация подсистемы поездок и участников	24
4.2.1	Реализация на клиенте	24
4.2.2	Реализация на сервере	24
4.2.3	Пользовательский поток (владелец/участник)	24
4.2.4	Результат для пользователя	24
4.3	Реализация подсистемы идей, обсуждений и маршрута	25
4.3.1	Реализация на клиенте	25
4.3.2	Реализация на сервере	26
4.3.3	Пользовательский поток (владелец/участник)	26
4.3.4	Результат для пользователя	26
4.4	Реализация подсистемы расходов и балансов	27
4.4.1	Реализация на клиенте	27
4.4.2	Реализация на сервере	27
4.4.3	Пользовательский поток (владелец/участник)	28
4.4.4	Результат для пользователя	28
4.5	Реализация подсистем погоды, AI-подсказок и уведомлений	29
4.5.1	Реализация на клиенте	29
4.5.2	Реализация на сервере	30
4.5.3	Пользовательский поток (владелец/участник)	30
4.5.4	Результат для пользователя	30
4.6	Реализация офлайн-режима и синхронизации	30
4.7	Проверка сквозных пользовательских сценариев	30
4.8	Автоматизированное тестирование и анализ покрытия	33
4.8.1	Пошаговый запуск проверок (для внешнего проверяющего)	33
4.8.2	Методика расчета коэффициента покрытия	34
4.8.3	Контур файлов и причины исключений из расчета покрытия	34
4.8.4	Фактические результаты	36
4.8.5	Интерпретация результатов	36
4.9	Выводы по главе	37
5	Заключение	38
	Список источников	39

Реферат

Выпускная квалификационная работа посвящена разработке Android-приложения для совместного планирования поездок. Актуальность исследования обусловлена тем, что в реальной практике групповой подготовки поездки пользователи распределяют задачи между несколькими сервисами, что приводит к дублированию данных и потере контекста решений.

Объектом исследования является процесс совместного планирования поездок в малых группах пользователей. Предметом исследования выступают пользовательские сценарии, функциональные требования и проектные решения мобильного приложения, объединяющего ключевые этапы планирования в едином цифровом пространстве.

Цель работы состоит в проектировании и реализации мобильного приложения, обеспечивающего целостный сценарий группового планирования поездки. В рамках работы проведен анализ предметной области и существующих решений, сформулирован функциональный разрыв, разработаны архитектурные и модульные решения, а также описана программная реализация подсистем приложения.

Результатом работы является структурированная модель приложения, включающая управление поездками и участниками, работу с идеями и маршрутом, учет расходов и балансов, погодный контекст, AI-предложения, настройки уведомлений, а также офлайн-режим с последующей синхронизацией данных.

Практическая значимость работы заключается в применимости полученного результата для повседневного группового планирования поездок и в возможности дальнейшего развития системы в рабочей среде.

Ключевые слова: Android-приложение, совместное планирование поездок, маршрут по дням, учет расходов, баланс участников, офлайн-синхронизация, AI-предложения.

Abstract

This graduation thesis is focused on developing an Android application for collaborative trip planning. The study is relevant because in real group planning practice users typically split their work across multiple digital services, which causes data duplication and loss of decision context.

The object of the study is the collaborative trip planning process in small user groups. The subject of the study is user scenarios, functional requirements, and design decisions for a mobile application that combines key planning stages within a single digital workspace.

The goal of the thesis is to design and implement a mobile application that supports an end-to-end group trip planning workflow. The work includes domain analysis and review of existing solutions, identification of a functional gap, development of architectural and modular solutions, and description of software implementation.

The resulting artifact is a structured application model that covers trip and participant management, ideas and itinerary planning, expense tracking and balance calculation, weather context, AI-based suggestions, notification settings, and offline mode with subsequent synchronization.

The practical significance lies in the applicability of the proposed solution to everyday collaborative trip planning and in its readiness for further system evolution in an operational environment.

Keywords: Android application, collaborative trip planning, day-by-day itinerary, expense tracking, participant balance, offline synchronization, AI suggestions.

Основные определения, термины и сокращения

Поездка — сущность планирования, объединяющая даты, участников, идеи, маршрут и расходы.

Участник — пользователь, присоединенный к поездке и участвующий в совместных действиях.

Владелец — пользователь, который создал поездку и управляет доступом и модерацией.

Идея — пользовательское предложение активности или места до включения в маршрут.

Комментарий идеи — сообщение пользователя в обсуждении конкретной идеи.

Активность — элемент маршрута, назначенный на конкретный день поездки.

Расход — запись о планируемой или уже произведенной трате в рамках поездки.

Баланс — агрегированное представление финансовых обязательств участников поездки.

Офлайн-синхронизация — механизм локального сохранения действий при отсутствии сети с последующей отправкой изменений на сервер.

AI-предложение — автоматически сгенерированная рекомендация, которую пользователь может сохранить в список идей.

Локализация UI — отображение интерфейса на языке, выбранном политикой локализации приложения.

ТЗ — техническое задание.

ПМИ — программа и методика испытаний.

ТП — текст программы.

Deep link — ссылка, открывающая конкретный экран мобильного приложения.

CI (Continuous Integration) — практика непрерывной интеграции изменений в общий репозиторий с автоматическим запуском проверок сборки и тестов.

1 Введение

Планирование групповых поездок в повседневной практике чаще всего выполняется в нескольких цифровых каналах одновременно: обсуждения ведутся в мессенджерах, маршрут фиксируется в картографических сервисах, расходы учитываются в отдельных приложениях, а погодные данные и справочная информация ищутся во внешних источниках. Такая фрагментация приводит к дублированию действий, потере контекста принятых решений и повышенной координационной нагрузке на организатора поездки [8].

Актуальность темы обусловлена потребностью в едином мобильном инструменте, который поддерживает полный цикл совместного планирования: от формулирования идей и обсуждений до формирования маршрута по дням и прозрачного учета расходов. Для малых групп пользователей критична не только доступность отдельных функций, но и их согласованная работа в общем сценарии [7][8].

Объектом исследования является процесс совместного планирования поездок в малых группах.

Предметом исследования являются пользовательские сценарии, функциональные требования и проектные решения мобильного приложения, обеспечивающего совместную работу участников поездки в едином информационном пространстве.

Цель выпускной квалификационной работы - разработка и реализация мобильного приложения, предназначенного для совместного планирования поездок и снижения издержек групповой координации.

Для достижения поставленной цели решаются следующие задачи:

1. Проанализировать предметную область и выявить ограничения фрагментированного подхода к планированию поездок.
2. Рассмотреть существующие решения и определить их функциональные границы в контексте совместного планирования.
3. Сформулировать функциональный разрыв и обосновать целевую нишу разрабатываемого приложения.
4. Спроектировать архитектуру системы, модель данных и ключевые пользовательские сценарии.
5. Обосновать выбор методов и средств реализации.
6. Разработать серверную часть приложения (backend), реализующую API, правила доступа и хранение данных.
7. Разработать клиентскую часть Android-приложения, реализующую ключевые пользовательские сценарии совместного планирования поездок.

8. Провести проверку сквозных пользовательских сценариев и зафиксировать результаты.

Практическая значимость работы состоит в создании прикладного Android-приложения, в котором объединены пользовательски значимые функции: управление поездкой, совместный доступ, работа с идеями и маршрутом, учет расходов и балансов, погодный контекст, AI-предложения, уведомления и поддержка офлайн-работы с последующей синхронизацией [7][9].

Выпускная квалификационная работа включает три главы. В первой главе рассматриваются предметная область и существующие решения, во второй - проектирование системы, в третьей - программная реализация и результаты проверки сквозных пользовательских сценариев.

2 Предметная область и существующие решения

2.1 Предметная область и актуальность

Совместное планирование поездки представляет собой последовательность взаимосвязанных действий: выбор направления и дат, формирование списка активностей, обсуждение альтернатив, распределение расходов и согласование итогового маршрута. На каждом этапе участники принимают коллективные решения, которые влияют на дальнейшие шаги. Изменение одного параметра поездки, например даты или состава участников, затрагивает сразу несколько подсистем планирования.

Ключевая проблема предметной области заключается в разрыве контекста между инструментами, когда обсуждения, маршрут и расходы фиксируются в разных сервисах. В результате участники получают несколько версий одного и того же плана, а владелец поездки вынужден вручную синхронизировать информацию. Это увеличивает вероятность ошибок, усложняет контроль договоренностей и снижает прозрачность процесса [8].

Таким образом, актуальной является задача создания единого мобильного пространства, в котором данные поездки связаны на уровне пользовательского сценария, а переход между этапами планирования выполняется без потери контекста [7][8][9].

2.2 Описание существующих решений

Для анализа использованы решения, представленные в проектном предложении, а также их публичные описания [8]. Рассматриваемые сервисы закрывают отдельные аспекты группового планирования и позволяют определить функциональные границы существующих подходов.

2.2.1 Splitwise

Splitwise ориентирован на учет совместных расходов и расчет взаиморасчетов между участниками группы [1]. Сервис предоставляет удобные механизмы фиксации трат и контроля задолженности, однако не покрывает задачи обсуждения идей и формирования маршрутного плана по дням в рамках единого процесса [1][8].

2.2.2 Wanderlog

Wanderlog предназначен для планирования поездок и совместного редактирования маршрута [2]. Сильной стороной решения является маршрутная часть, однако в контексте целевой постановки разрабатываемого приложения финансовые и дискуссионные сценарии представлены ограниченно и не формируют целостный процесс принятия решений [2][8].

2.2.3 TravelSpend

TravelSpend специализируется на фиксации расходов во время поездки [3]. Продукт эффективен для финансового учета, но не позиционируется как единая среда, объединяющая идеи, обсуждения, маршрут и совместное управление доступом [3][8].

2.2.4 TripIt

TripIt решает задачу агрегации информации о поездке и автоматизации отдельных организационных операций [4]. При этом сервис не ориентирован на полноценную коллективную модерацию идей и их управляемый перенос в маршрут в рамках одного пользовательского пространства [4][8].

2.2.5 Яндекс Путешествия

В публикации от 3 апреля 2025 года для сервиса «Яндекс Путешествия» описан режим совместной поездки по ссылке-приглашению [5]. Данный функционал улучшает совместный доступ к информации, однако основной акцент сервиса остается на сценариях подбора и бронирования, а не на полном цикле группового планирования с интегрированным учетом обсуждений и расходов [5][8].

2.2.6 RUSSPASS

RUSSPASS ориентирован на подбор маршрутов и туристический контент [6]. По открытым описаниям сервис поддерживает информационные и навигационные возможности путешествия, но не заявляет полный набор механизмов коллективного планирования, включая управляемые обсуждения и баланс расходов в рамках одной рабочей области [6][8].

Проведенный обзор показывает, что существующие продукты преимущественно специализируются на отдельных задачах. Наиболее частый подход - закрытие одного функционального сегмента (маршрут, расходы или контент), а не сквозного сценария совместной подготовки поездки.

2.3 Описание разрабатываемого решения

Разрабатываемое решение направлено на объединение ключевых пользовательских действий в рамках одной поездки как центральной сущности. Функциональная модель сервиса включает:

1. Авторизацию и персональный профиль пользователя.
2. Создание и управление поездками с параметрами дат, валюты и состава участников.
3. Механизм приглашений для совместного доступа.

4. Подсистему идей и обсуждений с ролями владельца и участника.
5. Маршрут по дням с операциями добавления, переноса и переупорядочивания активностей.
6. Учет расходов с балансом и статусами оплаты.
7. Погодный модуль по контексту поездки.
8. AI-подсказки с сохранением предложения в список идей.
9. Настройки уведомлений и поддержку офлайн-работы с последующей синхронизацией [7][9].

С точки зрения пользовательского результата разрабатываемое приложение формирует единую рабочую область, в которой обсуждение и фиксация решений не разрываются между внешними сервисами. Это позволяет уменьшить количество повторных действий и повысить прозрачность группового планирования [7][8].

2.4 Сравнительный анализ существующих решений

Для сопоставления решений использована матрица критериев, непосредственно связанных с целевой задачей разрабатываемого приложения.

Таблица 1: Сравнение существующих решений и разрабатываемого приложения

Сервис	Совместное планирование в одном пространстве	Идеи и обсуждения	Маршрут по дням	Расходы и баланс	Погода	AI-подсказки	Приглашения / совместный доступ
Splitwise	Нет	Нет	Нет	Да	Нет	Нет	Да
Wanderlog	Нет	Нет	Да	Нет	Нет	Нет	Да
TravelSpend	Нет	Нет	Нет	Да	Нет	Нет	Нет
TripIt	Нет	Нет	Да	Нет	Нет	Нет	Нет
Яндекс Путешествия	Нет	Нет	Нет	Нет	Нет	Нет	Да
RUSSPASS	Нет	Нет	Нет	Нет	Нет	Нет	Нет
Разрабатываемое приложение	Да	Да	Да	Да	Да	Да	Да

Примечание: в матрице используются только бинарные оценки. Если функция в стороннем сервисе присутствует фрагментарно и не закрывает критерий целиком, в таблице фиксируется значение “Нет”.

Результаты сравнительного анализа показывают, что у рассмотренных решений присутствуют сильные стороны в отдельных функциональных направлениях, однако в рамках выбранных критериев и источников [1][2][3][4][5][6] не прослеживается целостное покрытие сквозного сценария «идея -> обсуждение -> маршрут -> расходы -> синхронизация» в одном пользовательском пространстве [8].

2.5 Функциональный разрыв и обоснование ниши разрабатываемого приложения

По итогам анализа можно выделить функциональный разрыв, релевантный предметной области совместного планирования поездок:

1. Отсутствие единого пространства, где одновременно поддерживаются обсуждение идей, маршрутное планирование и финансовая прозрачность.
2. Недостаточная связность между этапами принятия решения и фактическим включением активности в маршрут.
3. Ограниченная поддержка коллективной работы при изменениях данных и нестабильном подключении.

Позиционирование разрабатываемого приложения определяется как интегрированный мобильный сервис для небольших групп пользователей, которым необходима единая среда принятия и фиксации решений по поездке. В отличие от специализированных решений, приложение ориентировано на целостный процесс, где каждая подсистема работает в контексте одной поездки и общей ролевой модели [7][8][9].

2.6 Выводы по главе

В первой главе рассмотрена предметная область совместного планирования поездок и обоснована актуальность ее цифровой интеграции. Проведен обзор существующих решений, показавший, что распространенные сервисы закрывают отдельные подзадачи, но в рамках принятых критериев сравнения не демонстрируют целостного сквозного сценария коллективной подготовки поездки.

На основе сравнительного анализа сформулирован функциональный разрыв и определено позиционирование разрабатываемого приложения как решения, объединяющего в едином пользовательском процессе управление поездкой, обсуждение идей, маршрут, расходы и вспомогательные сервисы. Полученные результаты задают основу для второй главы, в которой рассматриваются проектные решения и архитектура системы.

3 Проектирование системы

3.1 Пользовательские сценарии

Проектирование системы опирается на ролевую модель «владелец поездки - участник поездки» и на сквозные сценарии, формирующие целевую пользовательскую ценность. Владелец инициирует поездку и управляет доступом, участники участвуют в обсуждениях, планировании маршрута и распределении расходов.

Ключевые сценарии, учитываемые при проектировании, представлены в таблице.

Таблица 2: Ключевые пользовательские сценарии и риски

Сценарий	Компоненты	Результат	Риски
Создание поездки и приглашение участника	Клиент Android, API поездок, API приглашений	Формируется общая рабочая область поездки для владельца и участника	Невалидная или устаревшая ссылка приглашения
Идея → обсуждение → маршрут	Подсистема идей, новые комментарии real-time, модуль маршрута	Предложение участника после обсуждения фиксируется в плане дня	Конкурирующие изменения статуса идеи
Добавление расхода и пересчет баланса	Подсистема расходов, участники поездки, клиентский расчет итогов по данным API	Пользователи видят актуальные доли и итоговый баланс	Ошибки входных данных расхода
Погода и AI-предложение	Погодный модуль, AI-модуль, подсистема идей	Пользователь получает контекст и сохраняет предложение в идеи	Частичная недоступность внешних сервисов
Работа без сети и синхронизация	Локальное хранилище, очередь изменений, API синхронизации	Операции создания, изменения и удаления фиксируются локально и отправляются после восстановления сети	Коллизии при одновременных изменениях

Для демонстрации связности модулей в главе 2 используется базовый поток «идея -> обсуждение -> маршрут», поскольку именно он отражает переход от коллективного обсуждения к формальному плану поездки. Поток реализуется как последовательность проверяемых шагов: участник формирует идею, участники обсуждают ее в комментариях, затем по результату обсуждения идея либо переносится в маршрут по дням, либо остается в списке идей для последующей доработки.

3.2 Архитектура системы

Система проектируется как клиент-серверное приложение. Клиентская часть реализует пользовательские экраны, локальное хранение и координацию сценариев. Серверная часть централизует бизнес-правила, доступ к данным и интеграцию с внешними сервисами. Такой подход поддерживает единый источник актуального состояния поездки при сохранении отзывчивости интерфейса [12][13][14].

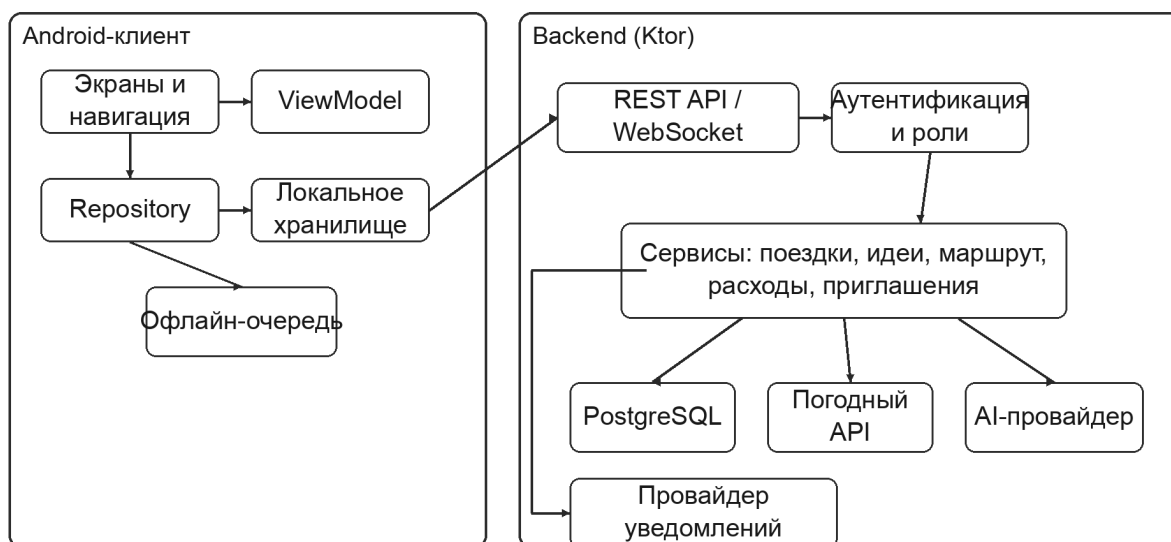


Рис. 3.1: обзорная архитектура системы Android client -> Backend -> PostgreSQL/интеграции.

Архитектурная схема фиксирует два уровня согласования данных: оперативный (онлайн-взаимодействие через API и каналы обновлений) и отложенный (офлайн-изменения с последующей синхронизацией). Такое разделение необходимо для мобильного сценария, в котором состояние сети не гарантировано [19].

3.2.1 Архитектура клиентской части

Клиент построен по многоуровневой схеме UI -> ViewModel -> Repository -> Data source.

1. Слой интерфейса включает экраны Jetpack Compose и навигацию между пользовательскими сценариями.
2. В слое представления используются **ViewModel** и потоки состояния (**StateFlow/SharedFlow**), что задает цикл «событие -> изменение состояния -> обновление экрана».
3. Репозитории инкапсулируют работу с REST API, локальным кэшем и очередью синхронизации, чтобы экран не зависел от источника данных напрямую.

4. Инфраструктурный слой включает внедрение зависимостей (Hilt) и фоновые задачи синхронизации (WorkManager), которые восстанавливают согласованность после появления сети.

Такое разделение снижает связанность экранов с сетевыми деталями и позволяет развивать клиентские сценарии по модулям (поездки, идеи, маршрут, расходы) без дублирования сетевой и кэш-логики [14][15][16][19].

3.2.2 Архитектура серверной части

Сервер реализован как модульный backend на Ktor с явным разделением ответственности:

1. Точка входа приложения инициализирует конфигурацию, подключение к базе данных, аутентификацию и общие плагины (логирование, сериализация, обработка ошибок, WebSocket).
2. HTTP-маршруты сгруппированы по предметным областям (`auth`, `trip`, `idea`, `itinerary`, `expense`, `invite`, `notification`, `sync`), что упрощает сопровождение API.
3. Предметные проверки и доступ к данным сосредоточены в серверных модулях и репозиториях, чтобы правила валидации и доступа не дублировались по маршрутам.
4. Интеграции с внешними сервисами (погода, AI, уведомления) вынесены в отдельные компоненты, изолированные от ядра предметной логики.

Для обсуждений используется отдельный WebSocket-канал: публикация нового комментария выполняется через событие в канале, при этом проверка прав участия в поездке остается на сервере [12][13][17][18].

3.2.3 Взаимодействие клиента и сервера

Взаимодействие строится в двух режимах:

1. синхронный режим: REST API для операций по поездкам, идеям, маршруту, расходам и настройкам;
2. событийный режим: WebSocket для новых комментариев и уведомления для фоновых оповещений.

Сервер выступает источником истинного состояния, а клиент поддерживает локальную копию данных и очередь изменений. При нестабильной сети пользователь продолжает работу с локальными данными, после восстановления связи накопленные операции отправляются на сервер и состояние приводится к согласованному виду [12][13][19].

3.3 Модель данных и связи сущностей

Концептуальная модель данных строится вокруг сущности поездки как корневого объекта. Все функциональные подсистемы связаны с поездкой и наследуют ее контекст (даты, участники, ограничения, валюту).

Основные связи сущностей описываются следующим образом:

1. Пользователь может участвовать в нескольких поездках, а одна поездка содержит нескольких участников.
2. Идея принадлежит конкретной поездке и может быть преобразована в активность маршрута.
3. Комментарии являются элементами обсуждения идеи и связаны с автором.
4. Маршрут представлен набором дней, каждый день содержит упорядоченный список активностей.
5. Расход относится к поездке и включает распределение долей между выбранными участниками.
6. Погодные данные привязываются к выбранному городу и датам поездки.
7. Предложение AI создается в контексте поездки и может быть сохранено как идея.

Для серверного хранения применяется реляционная модель данных на основе PostgreSQL. В текущей реализации структура таблиц инициализируется схемой проекта и поддерживается в кодовой базе. На стороне клиента сохраняется локальное состояние, необходимое для восстановления пользовательского процесса при временной потере сети [10][11][19].

3.4 Подсистемы и функциональное назначение

Для проектирования и последующей реализации выделены функциональные подсистемы, каждая из которых решает отдельную задачу, но работает в рамках общего сценария поездки.

Таблица 3: Подсистемы приложения и их назначение

Подсистема	Функции	Пользовательская ценность	Ограничения
Авторизация и профиль	Вход, управление профилем, выход, удаление аккаунта	Контролируемый доступ к персональным данным	Зависимость от доступности внешней аутентификации
Поездки и участники	Создание/редактирование поездки, управление участниками, приглашения	Быстрое формирование совместного пространства	Валидация дат и проверка корректности приглашений
Идеи и обсуждения	Создание идей, комментарии, модерация статуса	Прозрачная фиксация коллективных решений	Ролевые ограничения операций владельца/участника
Маршрут по дням	Добавление, перенос и упорядочивание активностей	Формирование исполнимого плана поездки	Ограничение действиями в диапазоне дат поездки
Расходы и баланс	Добавление расходов, сплиты, контроль статусов оплаты	Прозрачность финансовой ответственности	Валидность сумм, состава участников и валюты
Погода и AI-подсказки	Прогноз по городу поездки, генерация и сохранение предложений	Поддержка принятия решений по активности	Зависимость от внешних API и качества входного контекста
Настройки уведомлений	Управление категориями уведомлений	Снижение риска пропуска важных изменений	Ограничения платформенной доставки push
Офлайн и синхронизация	Локальная запись, отложенная отправка операций создания/изменения/удаления	Непрерывность работы при нестабильной сети	Конфликтные изменения при поздней синхронизации

Представленное разделение на подсистемы соответствует требованиям ТЗ и задает основу для модульной реализации, где пользователь наблюдает целостный процесс, а не набор изолированных экранов [7].

3.5 Выбор методов и средств реализации

Выбор методов и средств реализации выполнен по явным критериям: (1) поддержка мобильного сценария и офлайн-работы, (2) предсказуемость сопровождения, (3) совместимость со стеком Android/Backend, (4) стоимость доработок при расширении продук-

та.

3.5.1 Серверная часть

Серверная часть ориентирована на централизованную реализацию предметных правил и внешних интеграций. Выбран стек Kotlin + Ktor [12][13] по следующим причинам:

1. единый язык с клиентом снижает затраты на обучение и сопровождение команды;
2. Ktor предоставляет REST и WebSocket в одном фреймворке, что покрывает сценарии API и обсуждений;
3. стек допускает модульную структуру маршрутов и тестирование без сложной дополнительной инфраструктуры.

В качестве альтернатив рассматривались Java/Spring и Node.js/Express; для данного проекта выбран Kotlin/Ktor как более совместимый со стеком Android и требованиями текущего проекта.

Архитектурно на сервере принят паттерн «маршрут -> предметный модуль -> репозиторий». Такой подход выбран для того, чтобы валидация входных данных, проверка ролей и операции с базой данных проходили через единые точки принятия решений.

3.5.2 Хранилище данных

Для транзакционного хранения предметных сущностей используется PostgreSQL [10][11]. Выбор обусловлен:

1. реляционной моделью с явными связями между поездками, участниками, идеями, маршрутом и расходами;
2. транзакционными гарантиями ACID для операций, влияющих на совместное состояние группы;
3. воспроизводимым развертыванием через инициализационную схему проекта.

В качестве альтернатив рассматривались встраиваемые и документные хранилища; на текущем этапе они не обеспечивают такой же предсказуемости транзакционных операций для выбранной модели данных.

3.5.3 Клиентская часть

Мобильный клиент реализуется для платформы Android на Kotlin с Jetpack Compose, Hilt и Retrofit [14][15][16]. Выбор обоснован:

1. Compose — декларативное управление состоянием сложных экранов и быстрые итерации UI;

2. Hilt — управляемая сборка зависимостей для модульных экранов;
3. Retrofit/OkHttp — типизированные клиентские контракты и стандартный стек сетевого взаимодействия.

Альтернативный XML-based UI для текущего состава экранов не выбран из-за большей стоимости поддержки состояния и изменений интерфейса.

В клиентской архитектуре зафиксирована связка паттернов MVVM + Repository + DI:

1. MVVM изолирует состояние экрана в `ViewModel` и делает поведение экрана воспроизводимым;
2. Repository скрывает различия между сетевым источником, локальным кэшем и отложенной синхронизацией;
3. DI (Hilt) снижает связанность модулей и упрощает модульное тестирование.

3.5.4 Интеграционные компоненты

Для расширения пользовательского процесса в проекте используются внешние сервисы прогноза погоды и AI-генерации. В подсистеме офлайн-работы применяется механизм фоновой обработки задач синхронизации. Такой выбор позволяет сохранить работоспособность ключевых сценариев даже при частичной недоступности внешних источников [17][18][19].

Для интеграций используется адаптерный подход: вызовы внешних API инкапсулируются в отдельных компонентах, а прикладные модули работают с внутренними моделями. Это уменьшает объем изменений при замене провайдера и снижает риск каскадных правок в доменной логике.

3.5.5 Обоснование выбранных паттернов

Обоснование архитектурных паттернов в проекте сводится к проверяемым требованиям эксплуатации:

1. сценарии выполняются в нестабильной сети, поэтому требуется отдельная обработка онлайн- и офлайн-путей (локальный кэш + очередь + фоновая синхронизация);
2. в системе много экранов и форм, поэтому состояние интерфейса должно быть централизовано в `ViewModel`, а не распределено по компонентам UI;
3. предметные ограничения (роли, принадлежность к поездке, валидность приглашения) должны проверяться на сервере в единых точках, чтобы избежать расхождений между сценариями;

4. внешние сервисы могут быть частично недоступны или заменяться, поэтому интеграции изолируются адаптерным слоем;
5. модульная декомпозиция по подсистемам уменьшает стоимость локальных изменений и снижает риск регрессий при доработках.

3.6 Нефункциональные решения

Нефункциональные решения сформулированы с позиции эксплуатационной устойчивости и качества пользовательского опыта.

1. **Согласованность данных.** Сервер обеспечивает единые правила валидации и хранения, а также разрешение коллизий при синхронизации, чтобы все участники поездки видели согласованное состояние после операций создания, изменения и удаления [12][13].
2. **Надежность пользовательских сценариев.** Критические действия (идеи, маршруты, расходы) проектируются с учетом обработки ошибок и восстановления после сетевых сбоев [7][19].
3. **Безопасность доступа.** В пользовательских сценариях доступ к действиям внутри поездки ограничен аутентификацией и участием пользователя в поездке; ролевые операции владельца и участника разграничены на уровне API-маршрутов модерации и управления доступом [7][12].
4. **Интеграционная устойчивость.** При недоступности отдельных внешних сервисов базовые функции совместного планирования остаются доступными, а вспомогательные данные обновляются после восстановления интеграций [17][18].
5. **Эксплуатационная применимость.** Архитектура допускает развертывание серверной части в облачной среде и публикацию мобильного клиента в стандартном канале распространения Android-приложений [20][21].

Для проверки проектных решений, связанных с мобильной эксплуатацией, в текст включается отдельная иллюстрация офлайн-режима и восстановления согласованного состояния.



Рис. 3.2: офлайн-режим и синхронизация изменений: действие офлайн -> запись в очередь -> восстановление сети -> синхронизация.

3.7 Выводы по главе

Во второй главе выполнено проектирование системы на уровне пользовательских сценариев, архитектуры, модели данных и разделения на подсистемы. Показано, что выбранные проектные решения ориентированы на поддержку полного цикла совместного планирования поездки в едином мобильном приложении.

Сформированы проектные основания для реализации ключевых модулей: авторизации, управления поездкой и участниками, идей и обсуждений, маршрута, расходов, погодного и AI-контекста, уведомлений, офлайн-работы и синхронизации. Обоснован выбор методов и средств реализации, достаточный для перехода к этапу программной реализации, представленному в следующей главе.

4 Программная реализация системы

В данной главе представлено описание программной реализации системы и итогового пользовательского интерфейса. По структуре изложения глава построена по модульному принципу: для каждой функциональной подсистемы зафиксировано, что реализовано на клиенте, что реализовано на сервере, как выглядит пользовательский поток и какой результат получает пользователь.

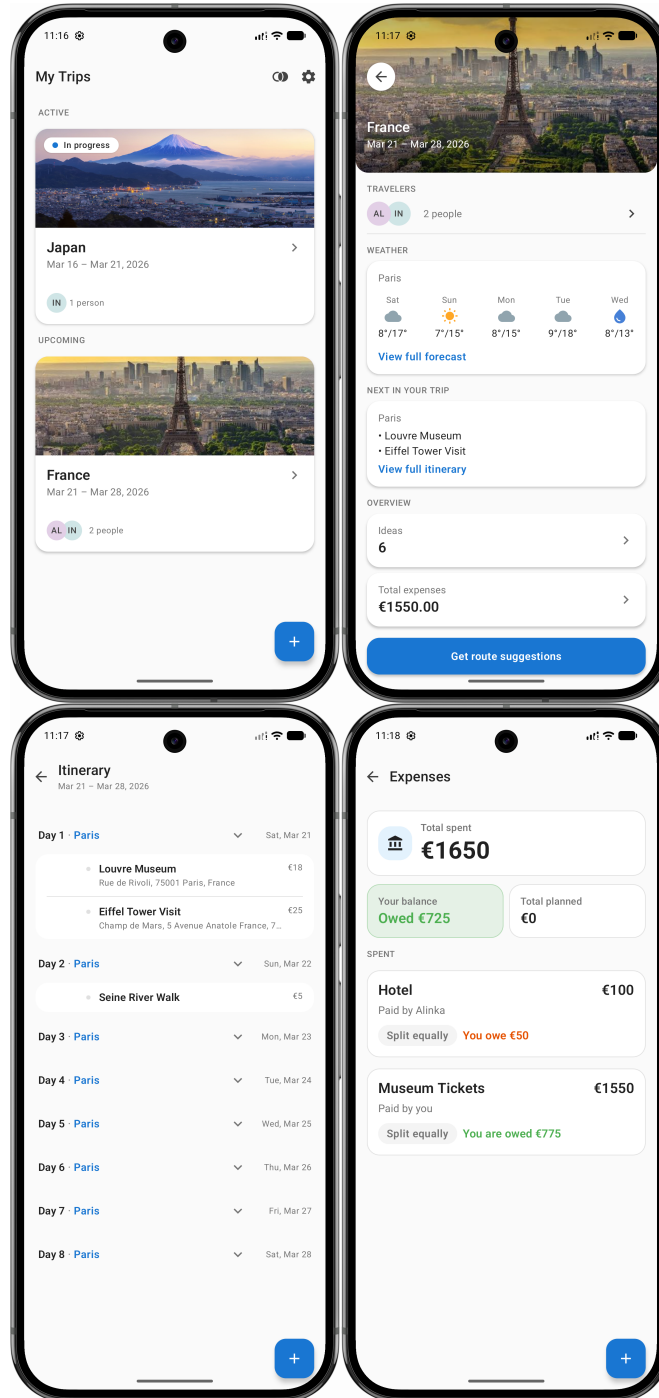


Рис. 4.1: Общий интерфейс приложения: основные экраны списка поездок, деталей поездки, идей, маршрута и расходов.

В качестве демонстрационного сценария рассматривается совместное планирование

поездки, где владелец создает поездку, приглашает участника, формирует идеи и маршрут, распределяет расходы, использует погодные данные и AI-подсказки. Такой выбор позволяет проверить не отдельные экраны, а целостность системы и связность реализованных модулей [7][8][9].

4.1 Реализация подсистемы авторизации и профиля

4.1.1 Реализация на клиенте

На стороне Android реализован экран входа и связанный поток инициализации сессии. Пользователь проходит авторизацию и после успешного входа попадает в рабочий режим приложения. В клиентской части также реализован экран настроек, где доступны изменение имени, фотография профиля, завершение сессии и удаление учетной записи в предусмотренном пользовательском сценарии [7][9][14]. В интерфейсе применена политика локализации: при системной локали `ru` приложение отображает русский UI, в остальных локалях — английский UI.

Визуально подсистема построена так, чтобы отделять аутентификационный этап от этапа рабочей навигации. До входа пользователю доступен только поток авторизации; после входа активируется доступ к поездкам и остальным модулям. Такой переход упрощает модель доступа и снижает риск ситуации, когда неавторизованный пользователь может открыть внутренние экраны приложения.

4.1.2 Реализация на сервере

На серверной стороне реализованы маршруты аутентификации и сервисы выдачи токенов, включая проверку входных учетных данных и управление жизненным циклом сессии. Сервер отвечает за валидацию входа и разграничение доступа к защищенным маршрутам, связанным с поездками и пользовательскими данными [12][13].

Серверная реализация поддерживает продление/обновление сессии в рамках настроенной политики, что позволяет сохранять предсказуемый пользовательский опыт при длительной работе с приложением. Для операций профиля используются защищенные маршруты, требующие валидного контекста пользователя.

4.1.3 Пользовательский поток (владелец/участник)

1. Пользователь выполняет вход в приложении.
2. Система проверяет учетные данные и создает активную сессию.
3. Пользователь открывает настройки и изменяет профильные данные.
4. Обновленные данные сохраняются и отражаются в интерфейсе.

4.1.4 Результат для пользователя

Пользователь получает контролируемый доступ к данным поездки, персонализированный профиль и понятный сценарий управления учетной записью. На уровне взаимодействия это снижает риск ошибок доступа и повышает доверие к системе как к личному рабочему пространству.

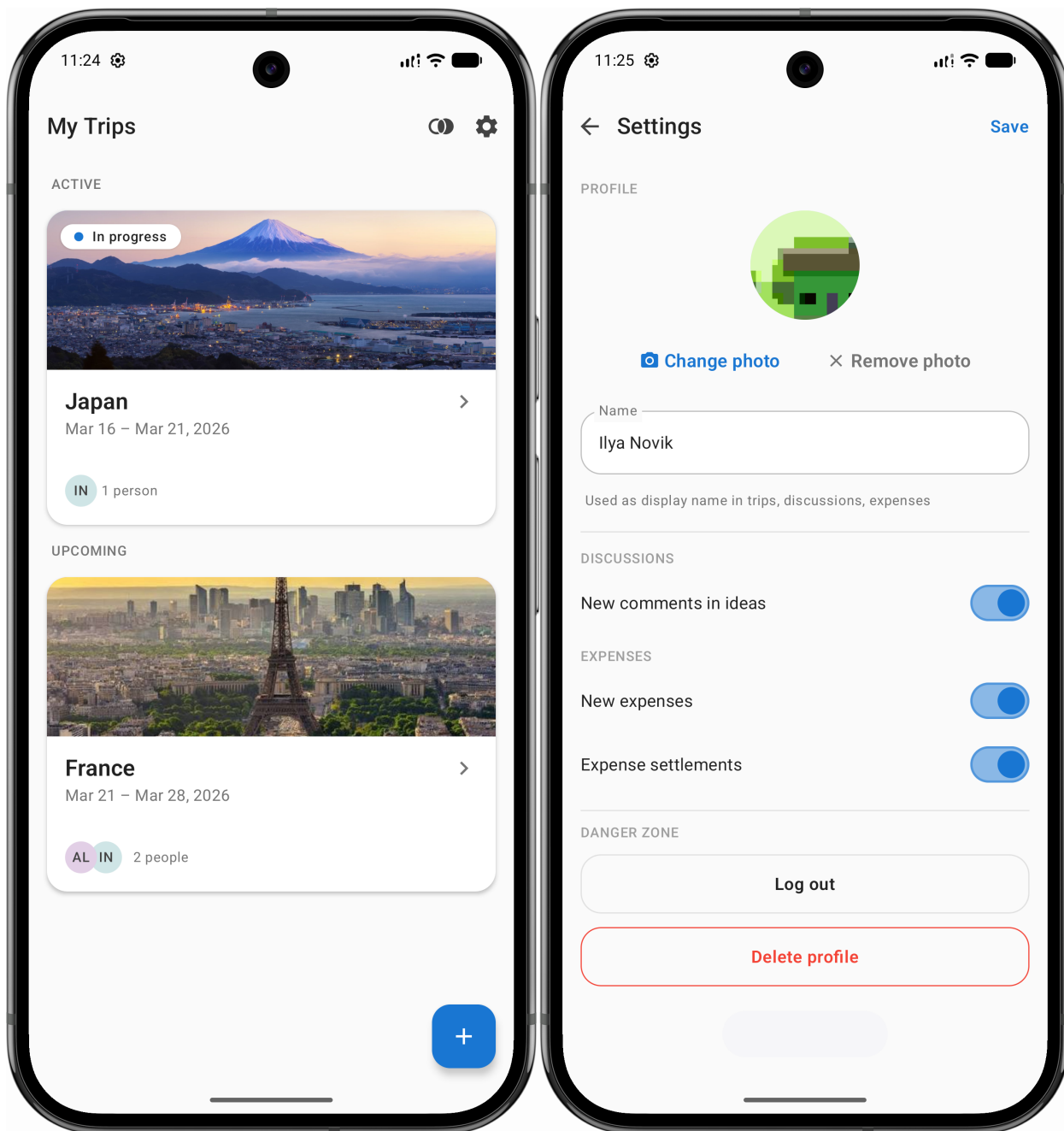


Рис. 4.2: Авторизация и профиль: экран успешного входа и экран настроек после изменения данных.

4.2 Реализация подсистемы поездок и участников

4.2.1 Реализация на клиенте

Клиентская часть включает экраны списка поездок, формы создания и редактирования поездки, карточку поездки и экран участников. В формах реализованы пользовательские валидаторы дат, названия и базовых параметров поездки, включая валюту и описание. В интерфейсе владельца доступны действия по приглашению участников и управлению составом поездки [7][9].

Отдельное внимание уделено представлению поездки как центра контекста. Пользователь сначала работает со списком поездок, затем открывает детали конкретной поездки и уже из нее переходит в подсистемы идей, маршрута, расходов и уведомлений. Такая навигация уменьшает фрагментацию и делает переходы между задачами последовательными.

4.2.2 Реализация на сервере

Сервер реализует маршруты создания, изменения, удаления и выдачи поездок, а также маршруты управления участниками и приглашениями. Для приглашений предусмотрены операции генерации и обработки ссылок, а также проверки валидности приглашения при присоединении. Действия с доступом участников выполняются с учетом роли пользователя в поездке [12][13].

Серверная часть обеспечивает согласованность состава участников: один и тот же участник не дублируется в рамках одной поездки, а невалидные приглашения не переводят пользователя в состояние «частично присоединен».

4.2.3 Пользовательский поток (владелец/участник)

1. Владелец создает поездку и задает параметры.
2. Владелец формирует приглашение и отправляет ссылку.
3. Участник открывает ссылку и присоединяется к поездке.
4. Оба пользователя видят общую поездку и единый список участников.

4.2.4 Результат для пользователя

Группа получает единое рабочее пространство поездки и прозрачный механизм совместного доступа. Это снимает потребность в ручной «сборке команды» вне приложения и сразу переводит участников в общий контекст планирования.

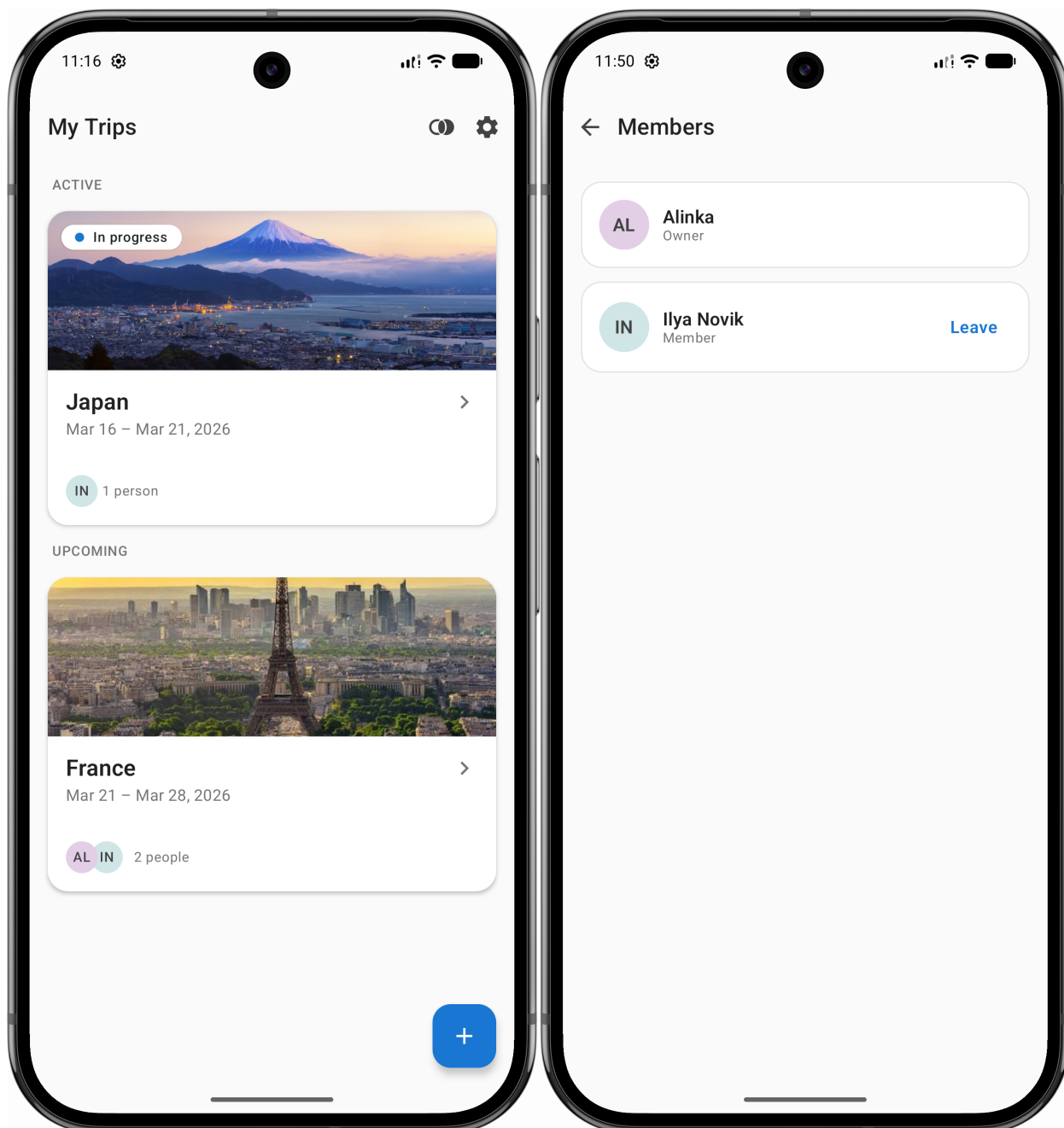


Рис. 4.3: Поездки и участники: создание поездки и состав участников после присоединения по ссылке.

4.3 Реализация подсистемы идей, обсуждений и маршрута

4.3.1 Реализация на клиенте

На клиенте реализованы экраны списка идей, формы создания/редактирования, карточка идеи с обсуждением, а также экран маршрута по дням. Для обсуждений используется интерфейс, поддерживающий поступление новых сообщений в реальном времени, что критично для совместного принятия решения. Для маршрута реализованы добавление активностей, изменение порядка, перенос между днями и сценарии обработки дней, выходящих за актуальный диапазон дат поездки [7][9].

Ключевой особенностью интерфейса является связка «идея -> решение -> вклю-

чение в маршрут». Пользователь не теряет контекст между обсуждением и планом: карточка идеи может быть преобразована в элемент маршрута и дальше участвует в ежедневном планировании поездки.

4.3.2 Реализация на сервере

Серверная часть включает маршруты для идей, комментариев и маршрута, а также канал real-time обновлений для обсуждений. Сервер обеспечивает хранение статусов идеи, валидацию операций модерации и согласованную запись изменений в маршрут. В текущей версии real-time канал гарантированно публикует события создания комментариев; удаление комментариев выполняется через отдельный API-маршрут с проверкой прав автора [12][13].

Такое разделение позволяет сохранить единое правило: проверка предметных ограничений выполняется на сервере, а клиент отвечает за удобство выполнения действий.

4.3.3 Пользовательский поток (владелец/участник)

1. Участник создает идею и публикует ее в поездке.
2. Участники обсуждают идею в комментариях.
3. Владелец принимает решение по идее (одобрение/отклонение).
4. Одобренная идея переносится в конкретный день маршрута.
5. Порядок активностей корректируется в соответствии с планом группы.

4.3.4 Результат для пользователя

Пользователь получает непрерывный сценарий от обсуждения к исполнимому маршруту. Это устраняет типичную проблему разрыва между «чатом с предложениями» и «финальным планом», которая характерна для разрозненных инструментов [7][8].

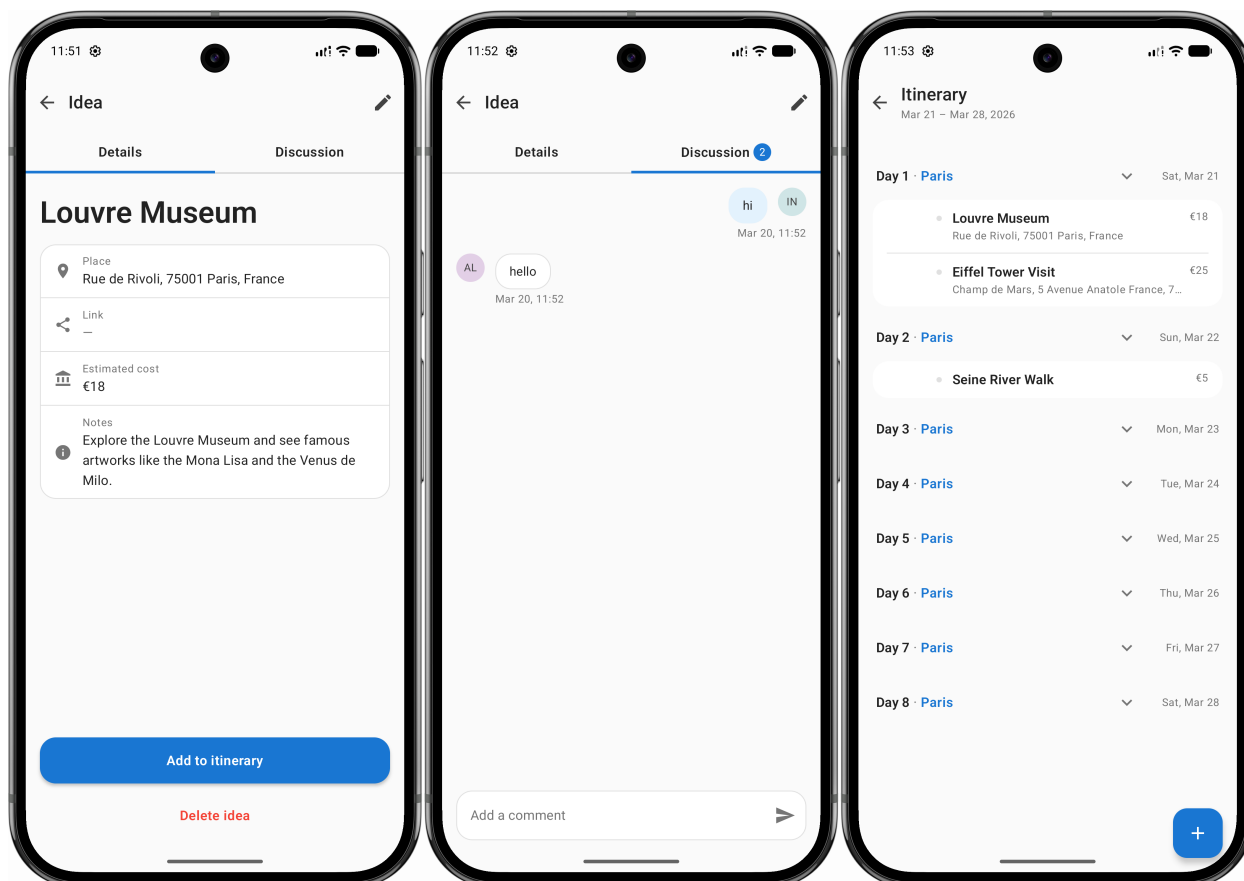


Рис. 4.4: Идеи, обсуждения и маршрут: сквозной поток от списка идей к обсуждению и добавлению в маршрут.

4.4 Реализация подсистемы расходов и балансов

4.4.1 Реализация на клиенте

Клиентская часть включает список расходов, форму создания/редактирования расхода, экран детализации и представление итогового баланса. Поддерживаются сценарии планируемого и оплаченного расхода, выбор плательщика, распределение долей и отметка факта оплаты участниками. Интерфейс ориентирован на то, чтобы пользователь видел не только отдельные записи, но и суммарный финансовый результат по поездке [7][9].

За счет прозрачного отображения долей и статусов оплаты снижается число уточняющих коммуникаций между участниками и упрощается закрытие финансовых обязательств по завершении поездки.

4.4.2 Реализация на сервере

Сервер реализует маршруты создания, изменения, удаления и получения расходов. В серверной логике выполняются проверки валидности суммы, участников и связки расхода с конкретной поездкой. Итоговые показатели для отображения баланса вычисляются на клиенте по данным расходов и долей участников, полученных через API [12][13].

4.4.3 Пользовательский поток (владелец/участник)

1. Пользователь создает расход с указанием суммы и состава участников.
2. Система рассчитывает доли и обновляет общую финансовую картину.
3. Участники отмечают оплату в рамках своих обязательств.
4. Баланс пересчитывается и отражается в интерфейсе.

4.4.4 Результат для пользователя

Группа получает прозрачную и проверяемую финансовую модель поездки. Пользователь видит, кто оплатил, кто должен и как меняется итоговый баланс при каждом действии.

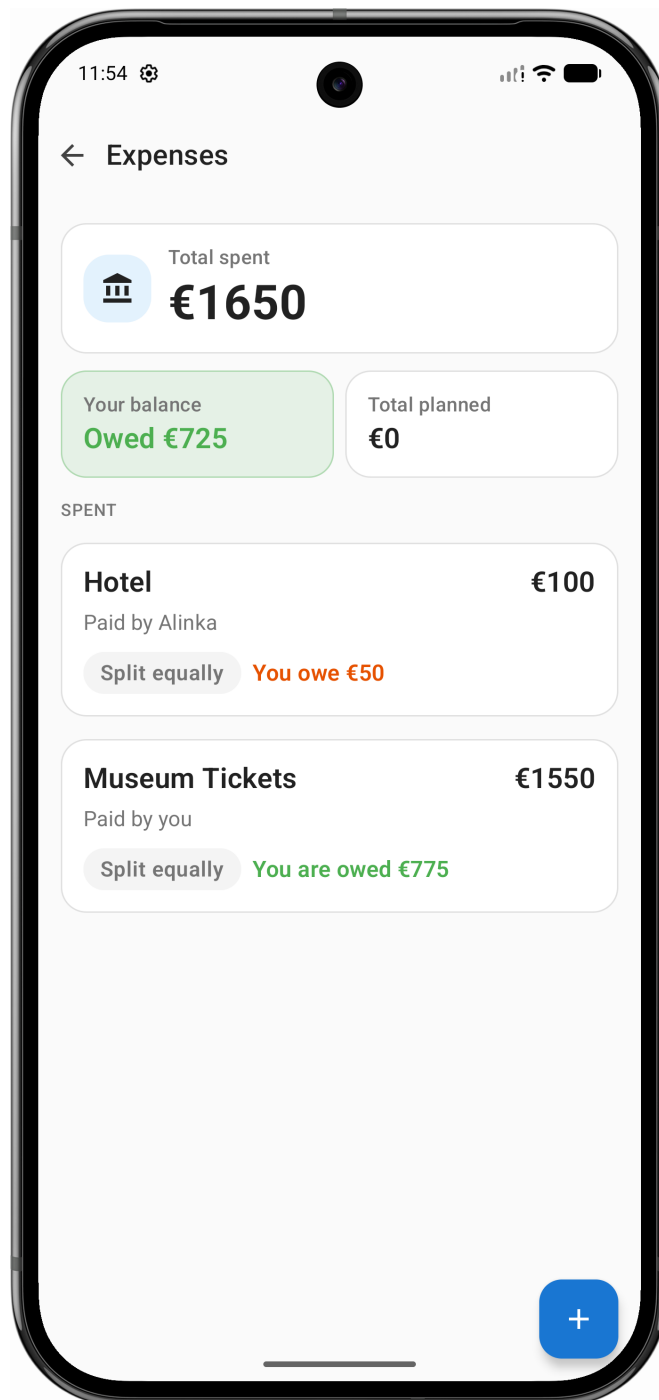


Рис. 4.5: Расходы и баланс: итоговые суммы и распределение долей участников.

4.5 Реализация подсистем погоды, AI-подсказок и уведомлений

4.5.1 Реализация на клиенте

На клиенте реализованы экран прогноза и погодные блоки в контексте поездки, интерфейс генерации AI-подсказок и экран настроек уведомлений. Пользователь может обновлять погодные данные, выбирать релевантный контекст поездки для AI-предложений и сохранять подходящие варианты в список идей. В настройках доступны переключатели категорий уведомлений [7][9].

Такая интеграция делает вспомогательные сервисы частью рабочего процесса, а не

внешним шагом «после планирования».

4.5.2 Реализация на сервере

Серверная часть интегрируется с погодным провайдером и AI-провайдером через выделенные маршруты и сервисные адаптеры. Для уведомлений реализованы маршруты регистрации токенов устройств и отправки событий. При этом погодные и AI-данные не заменяют основные функции поездки, а дополняют их контекстом принятия решений [17][18][20].

4.5.3 Пользовательский поток (владелец/участник)

1. Пользователь открывает погодный модуль поездки и обновляет прогноз.
2. Пользователь запрашивает AI-предложения для маршрута.
3. Подходящее предложение сохраняется в идеи поездки.
4. При включенных категориях уведомлений, выданном системном разрешении и доступном интернете пользователь получает уведомления по событиям идей (создание/комментарии) и расходам.

4.5.4 Результат для пользователя

Пользователь быстрее принимает решения по маршруту и при включенных уведомлениях, выданном разрешении и доступе к интернету получает актуальные события поездки без постоянной ручной проверки каждого экрана.

4.6 Реализация офлайн-режима и синхронизации

На текущем этапе офлайн-режим и синхронизация изменений в пользовательском контуре не реализованы в завершенном виде. Реализация данного функционального блока запланирована на следующий этап развития проекта.

4.7 Проверка сквозных пользовательских сценариев

В рамках верификации результата проведена функциональная проверка сквозных сценариев, отражающих ядро пользовательской ценности приложения. Проверка ориентирована на связность модулей и достижение пользовательского результата, а не только на изолированную работоспособность отдельных экранов.

Текущее покрытие модулей по реализованным возможностям представлено в таблице.

Таблица 4: Покрытие модулей приложения

Модуль	Реализованные возможности	Клиент/Сервер	Статус
Авторизация и профиль	Вход, изменение профиля, завершение сессии, удаление аккаунта в пользовательском потоке	Android, Backend	Реализовано
Поездки и участники	Создание/редактирование/удаление поездки, приглашение и присоединение	Android, Backend	Реализовано
Идеи и обсуждения	CRUD идей, статусы, новые комментарии в реальном времени	Android, Backend, WS	Реализовано
Маршрут	Планирование по дням, перенос и переупорядочивание активностей	Android, Backend	Реализовано
Расходы и баланс	Ведение расходов, сплиты, баланс и статусы оплаты	Android, Backend	Реализовано
Погода, AI, уведомления	Прогноз, генерация AI-предложений, настройки уведомлений	Android, Backend, внешние сервисы	Реализовано
Офлайн и синхронизация	Функциональность запланирована в следующих итерациях разработки	Android, Backend	Запланировано

Результаты проверки сквозных сценариев приведены в таблице ниже.

Таблица 5: Проверка сквозных пользовательских сценариев

Сценарий проверки	Шаги	Ожидаемый результат	Фактический результат
Создание поездки и приглашение участника	Владелец создает поездку, отправляет приглашение, участник присоединяется	У обоих пользователей отображается одна и та же поездка и общий состав участников	Сценарий воспроизводится, критических отклонений не зафиксировано
Идея → обсуждение → маршрут	Участник создает идею, новые комментарии поступают в реальном времени, идея переносится в маршрут	Идея из обсуждения становится элементом маршрута выбранного дня	Сценарий воспроизводится; результат подтвержден в рамках проведенной проверки
Управление расходом и балансом	Создается расход, задаются доли, фиксируется оплата	Баланс пересчитывается на клиенте по данным расходов и долей участников	Сценарий воспроизводится; несоответствий не обнаружено
Погода + AI-подсказка	Пользователь обновляет прогноз, генерирует AI-предложение, сохраняет в идеи	Предложение появляется в идеях, погодные данные отображаются по контексту поездки	Сценарий воспроизводится при доступности внешних сервисов
Настройки уведомлений	Пользователь изменяет категории уведомлений в настройках	При включенных категориях, выданном системном разрешении и доступном интернете система применяет выбранные категории (события идей и расходов)	Сценарий воспроизводится; применяются только подерживаемые категории

Проверка выполнялась как ручной сценарный прогон в тестовой среде приложения по набору тестовых сценариев. Представленные результаты отражают выполнение ключевых сценариев и не охватывают все возможные комбинации входных данных [7][8][9].

Проведенная проверка показывает, что реализованная система закрывает целевой набор пользовательских сценариев, сформулированный в ТЗ и обоснованный в аналитической части работы [7][8]. При этом дальнейшим этапом развития остается расши-

рение автоматизированного набора регрессионных тестов для сокращения стоимости последующих изменений.

4.8 Автоматизированное тестирование и анализ покрытия

Метрики раздела 3.8 используются как инженерный контроль процесса разработки (Android, Backend) и не заменяют клиентскую приемку по ПМИ, которая проверяет только пользовательские сценарии взаимодействия с приложением.

Для снижения регрессионных рисков в проекте внедрены автоматизированные модульные тесты и JVM-тесты пользовательского интерфейса (Compose/Robolectric), а также единые пороговые критерии качества по покрытию в клиентской и серверной частях [9].

4.8.1 Пошаговый запуск проверок (для внешнего проверяющего)

Чтобы воспроизвести результаты без знания внутренней структуры проекта, используется следующий порядок:

1. открыть корневую директорию репозитория, где находятся каталоги `android/`, `backend/` и скрипт `quality-check.sh`;
2. запустить единый сценарий проверки:

```
./quality-check.sh
```

3. при необходимости запускать модули отдельно:

```
# серверный модуль
cd backend && ./gradlew qualityCheck
```

```
# клиентский модуль Android
cd android && ./gradlew :app:qualityCheck
```

Сценарий `qualityCheck` в каждом модуле включает:

1. выполнение тестов;
2. формирование отчета JaCoCo в форматах XML и HTML;
3. проверку порогов покрытия строк и покрытия ветвлений.

Артефакты проверки находятся в фиксированных путях:

1. серверный модуль:
 - `backend/build/test-results/test/*.xml`;
 - `backend/build/reports/jacoco/test/jacocoTestReport.xml`;
 - `backend/build/reports/jacoco/test/html/index.html`.

2. клиентский модуль Android:

```
android/app/build/test-results/testDebugUnitTest/*.xml;  
android/app/build/reports/jacoco/jacocoDebugUnitTestReport/jacocoDebug  
UnitTestReport.xml;  
android/app/build/reports/jacoco/jacocoDebugUnitTestReport/html/index.  
html.
```

Критерий прохождения: в каждом модуле одновременно выполняются пороги покрытия строк не ниже 50% и покрытия ветвлений не ниже 40%. При нарушении порога задача Gradle завершается ошибкой, что позволяет применять эту проверку в CI.

4.8.2 Методика расчета коэффициента покрытия

В работе используется JaCoCo. Для каждого счетчика рассчитывается:

$$K_{\text{покрытия}} = \frac{N_{\text{покрыто}}}{N_{\text{покрыто}} + N_{\text{непокрыто}}} \cdot 100\%$$

Ключевые метрики:

1.

$$K_{\text{строк}} = \frac{N_{\text{покрытые строки}}}{N_{\text{покрытые строки}} + N_{\text{непокрытые строки}}} \cdot 100\%$$

2.

$$K_{\text{ветвлений}} = \frac{N_{\text{покрытые ветви}}}{N_{\text{покрытые ветви}} + N_{\text{непокрытые ветви}}} \cdot 100\%$$

Где:

1. $N_{\text{покрытые строки}}$ — строки, для которых в ходе теста выполнена хотя бы одна инструкция;
2. $N_{\text{непокрытые строки}}$ — строки, для которых не выполнено ни одной инструкции;
3. $N_{\text{покрытые ветви}} / N_{\text{непокрытые ветви}}$ — покрытые/непокрытые ветви условных конструкций.

4.8.3 Контур файлов и причины исключений из расчета покрытия

Покрытие считается не по всему бинарному артефакту, а по целевой части бизнес-логики, заданной в Gradle-конфигурации [9]. Такой подход нужен, чтобы метрика отражала качество тестирования бизнес-логики, а не технической обвязки.

Для клиентского модуля Android в расчет включаются скомпилированные классы `nvk/cotrip/**` (сборка `debug`), при этом исключаются:

1. сгенерированные и инфраструктурные классы:

```
R, BuildConfig, Manifest;
```

`*_Factory, *_MembersInjector, *_Impl;`
`DI/Hilt/Dagger.`

2. обвязка пользовательского интерфейса и точки входа (`*ScreenKt*, MainActivity, CoTripApp`);
3. отдельные инфраструктурные директории:
`di/**, ui/theme/**, data/network/dto/**.`
4. тестовые и синтетические классы:
`*Test*`, часть шаблонов `*$*`.

Для серверного модуля покрытие считается по классам основного набора (`main`) с исключением инфраструктурных слоев:

1. точки входа и загрузочная инициализация (`ApplicationKt`, часть подсистемы журналирования и конфигурации);
2. маршруты и транспортный слой (`routes/**, plugins/**`);
3. инфраструктура БД (`db/**, DatabaseFactory`);
4. часть внешних адаптеров доставки событий:
`FirebasePushService, CommentEventsPublisher, CommentsHub.`
5. служебные/сгенерированные вспомогательные конструкции Kotlin:
`$Companion, $serializer, $WhenMappings, $inlined$.`

Причины исключения указанных групп кода:

1. сгенерированный код не содержит проектных решений и может существенно «шуметь» в метрике, искажая сравнение между итерациями;
2. код начальной инициализации и внедрения зависимостей в основном выполняет связывание компонентов, а не реализует прикладные правила предметной области;
3. транспортный слой и внешние адаптеры проверяются преимущественно интеграционными и сценарными проверками, где приоритет имеет корректность контракта в целом, а не покрытие строк отдельных оберток;
4. исключение синтетических конструкций Kotlin предотвращает техническое занижение покрытия на коде, который разработчик напрямую не пишет.

Исключение из расчета покрытия не означает отсутствия проверки: часть таких компонентов контролируется косвенно через интеграционные тесты и сквозные пользовательские сценарии (раздел 3.7). Метрика покрытия в этой модели используется как индикатор качества автоматизированных тестов бизнес-логики.

4.8.4 Фактические результаты

Количественные показатели получены по результатам запуска `./quality-check.sh` от 18.03.2026 в текущем состоянии репозитория.

Таблица 6: Результаты автоматизированного тестирования и покрытия

Модуль	XML-отчеты	Тесты	Пропуски	Покрыв- аемые строк	Покрыв- аемые ветвлений	Пороговый кон- троль
Клиент Android	61	291 (0 ошибок)	0	3559/5677 (62.69%)	1597/3918 (40.76%)	Пройден
Сервер	10	46 (0 ошибок)	0	534/968 (55.17%)	280/577 (48.53%)	Пройден
Итог (сум- ма)	71	337 (0 ошибок)	0	4093/6645 (61.60%)	1877/4495 (41.76%)	Пройден

Пороговые значения в обоих модулях: покрытие строк не ниже 50%, покрытие ветвлений не ниже 40% (оба условия выполнены).

Дополнительно по объектам JaCoCo в расчетной части:

1. клиент Android: 32 пакета, 223 исходных файла, 192 класса в расчетной части (153 покрыто полностью/частично, 39 не покрыто).
2. сервер: 7 пакетов, 14 исходных файлов, 66 классов в расчетной части (51 покрыто полностью/частично, 15 не покрыто).

4.8.5 Интерпретация результатов

Полученные значения подтверждают выполнение минимальных пороговых критериев качества в обоих модулях. При этом:

1. покрытие строк в обоих модулях превышает установленный порог и отражает приемлемый охват исполняемых сценариев;
2. покрытие ветвлений в Android-модуле находится близко к порогу, поэтому набор тестов по альтернативным ветвям условий и граничным состояниям нужно расширять;
3. по зафиксированному прогону пропуски отсутствуют, что упрощает интерпретацию регрессионных метрик при повторных запусках.

Таким образом, автоматизированная проверка дополняет ручную верификацию сквозных сценариев (раздел 3.7) и формирует воспроизводимый процесс контроля качества для последующих итераций разработки [9].

4.9 Выводы по главе

В третьей главе представлена программная реализация приложения в модульной форме, соответствующей структуре и стилю рассмотренных примеров ВКР. Для каждой подсистемы зафиксированы реализованные возможности на клиенте и сервере, а также итоговый пользовательский результат.

Показано, что ключевая ценность системы формируется на уровне сквозных сценариев: участники группы могут совместно пройти путь от создания поездки и обсуждения идей до построения маршрута, распределения расходов и работы в условиях нестабильной сети. Реализация вспомогательных модулей (погода, AI, уведомления) встроена в общий процесс планирования и усиливает его практическую применимость.

Результаты функциональной проверки по набору сценариев указывают на соответствие текущей версии приложения целевому сценарию совместного планирования, заявленному в предыдущих главах. Таким образом, после аналитического и проектного этапов в главах 1 и 2, в главе 3 получено практическое подтверждение применимости разработанной системы по результатам проведенной проверки.

5 Заключение

В выпускной квалификационной работе рассмотрена предметная область совместного планирования поездок и обоснована необходимость интеграции ключевых пользовательских сценариев в едином мобильном приложении.

В первой главе выполнен анализ существующих решений и выявлен функциональный разрыв между специализированными сервисами и потребностью пользователей в сквозном процессе планирования. Во второй главе сформированы проектные решения для системы: пользовательские сценарии, архитектурная модель, разделение на подсистемы, модель данных и нефункциональные требования. В третьей главе описана программная реализация модулей системы и представлены результаты проверки сквозных сценариев.

Полученные результаты подтверждают, что текущая версия приложения обеспечивает целевой пользовательский процесс: управление поездкой и участниками, работу с идеями и маршрутом, учет расходов и баланса, использование погодного и AI-контекста, а также поддержку офлайн-режима с последующей синхронизацией.

Дальнейшее развитие работы связано с расширением автоматизированного тестирования, развитием аналитики пользовательских сценариев и усилением эксплуатационной устойчивости при росте нагрузки.

Список источников

- [1] Splitwise. URL: <https://www.splitwise.com/> (дата обращения: 15.03.2026).
- [2] Wanderlog. URL: <https://wanderlog.com/> (дата обращения: 15.03.2026).
- [3] TravelSpend. URL: <https://travel-spend.com/> (дата обращения: 15.03.2026).
- [4] TripIt. URL: <https://www.tripit.com/> (дата обращения: 15.03.2026).
- [5] Яндекс. «Яндекс Путешествия упростили совместные поездки», 03.04.2025. URL: <https://yandex.ru/company/news/01-03-04-2025> (дата обращения: 15.03.2026).
- [6] RUSSPASS (Google Play). URL: <https://play.google.com/store/apps/details?id=ru.russpass.tourist> (дата обращения: 15.03.2026).
- [7] Техническое задание.
- [8] Проектное предложение.
- [9] Текст программы.
- [10] PostgreSQL Global Development Group. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/> (дата обращения: 15.03.2026).
- [11] Инициализационная схема базы данных проекта.
- [12] Ktor. Ktor Documentation. URL: <https://ktor.io/docs/welcome.html> (дата обращения: 15.03.2026).
- [13] JetBrains. Kotlin Documentation. URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 15.03.2026).
- [14] Android Developers. Jetpack Compose. URL: <https://developer.android.com/compose> (дата обращения: 15.03.2026).
- [15] Android Developers. Dependency Injection with Hilt. URL: <https://developer.android.com/training/dependency-injection/hilt-android> (дата обращения: 15.03.2026).
- [16] Square. Retrofit. URL: <https://square.github.io/retrofit/> (дата обращения: 15.03.2026).
- [17] OpenWeather. Weather API. URL: <https://openweathermap.org/api> (дата обращения: 15.03.2026).
- [18] Yandex Cloud. YandexGPT (Foundation Models). URL: <https://yandex.cloud/en/docs/ai-studio/concepts/generation/models> (дата обращения: 15.03.2026).
- [19] Android Developers. WorkManager. URL: <https://developer.android.com/topic/libraries/architecture/workmanager> (дата обращения: 15.03.2026).
- [20] Yandex Cloud. Compute Cloud Documentation. URL: <https://yandex.cloud/en/docs/compute/> (дата обращения: 15.03.2026).
- [21] Android Developers. Distribute to Google Play. URL: <https://developer.android.com/distribute/google-play> (дата обращения: 15.03.2026).